

AI coding for researchers

Why your session hit a wall — and what to do about it

Ken Krige · ITS Utrecht University

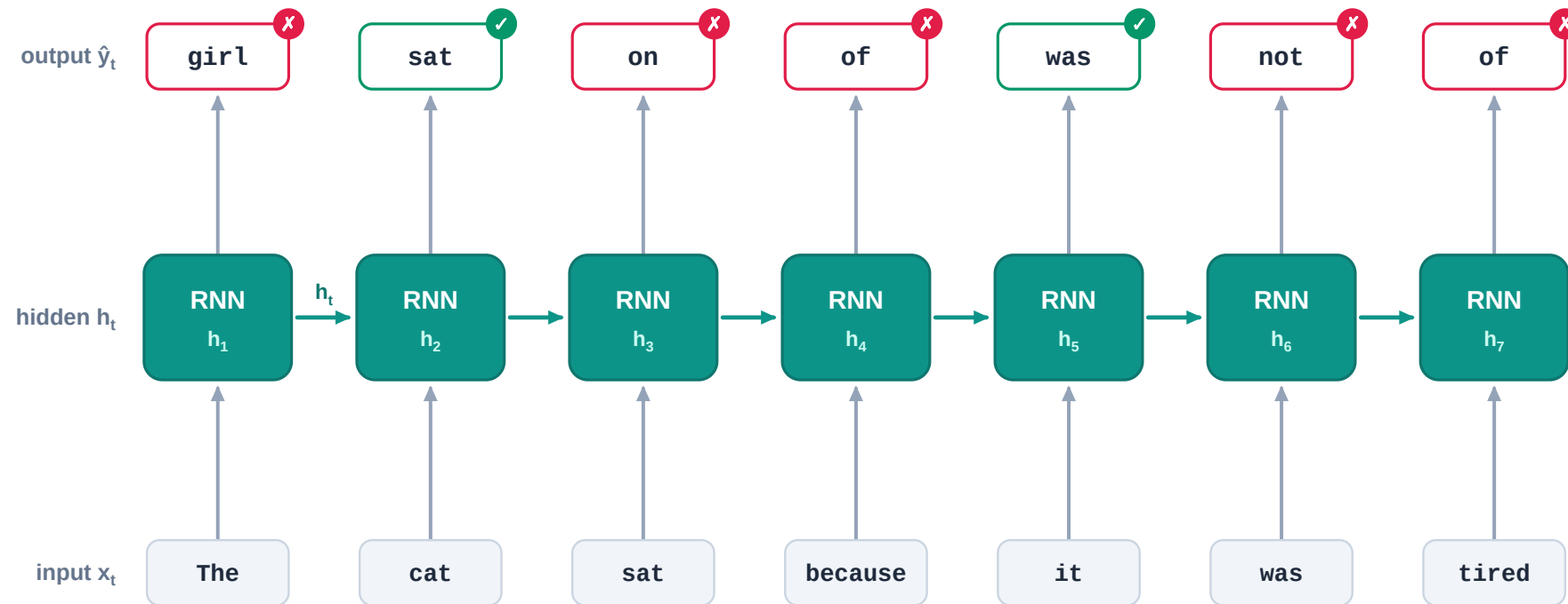
What I build with AI agents

Which Model?

- [Vellum LLM Leaderboard](#)
- [PinchBench](#)
- [WildClawBench](#)
- [LiveCodeBench](#)
- [OpenCompass LLM Leaderboard](#)
- [SonarSource — Coding Personalities of Leading LLMs](#)

Recurrent Neural Network (RNN)

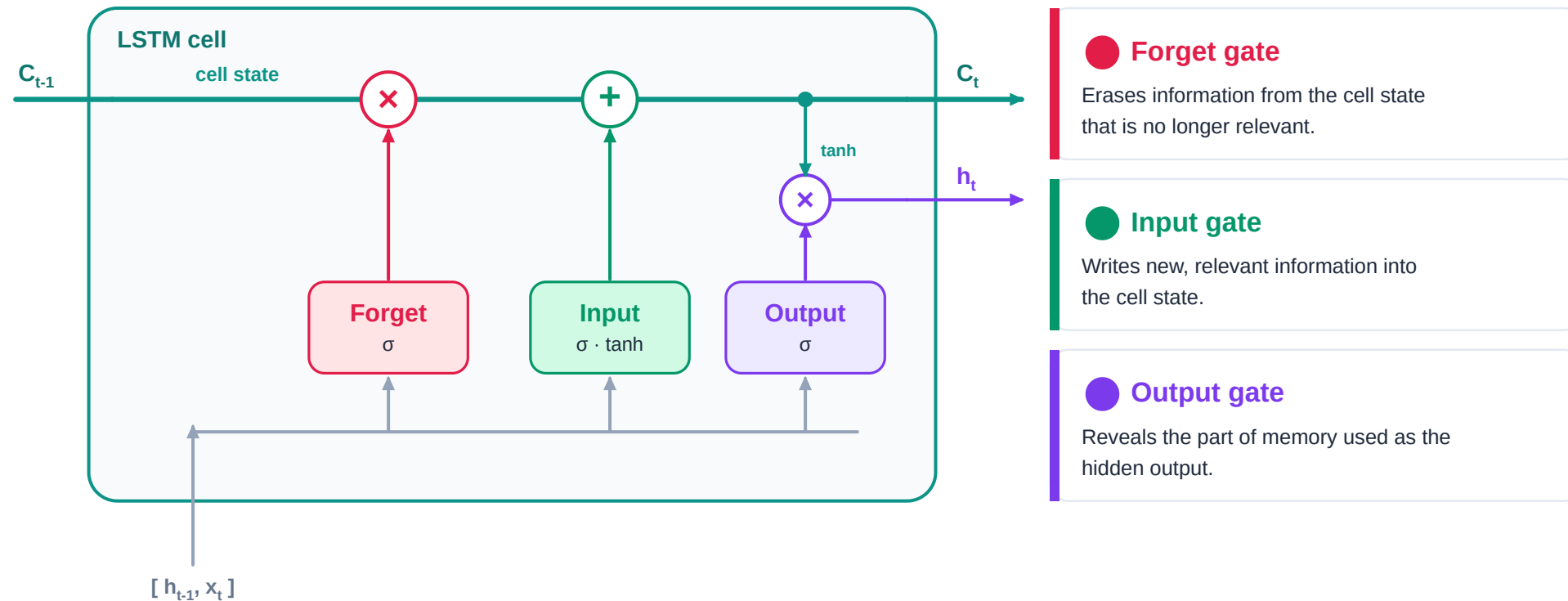
Reads a sequence one word at a time, carrying a hidden state (its memory) forward at each step.



- An RNN's next-word guesses are noisy and often wrong — here only 'sat' and 'was' land on the actual next word. And information from early words fades as the sequence grows — the vanishing-gradient problem.

Long Short-Term Memory (LSTM)

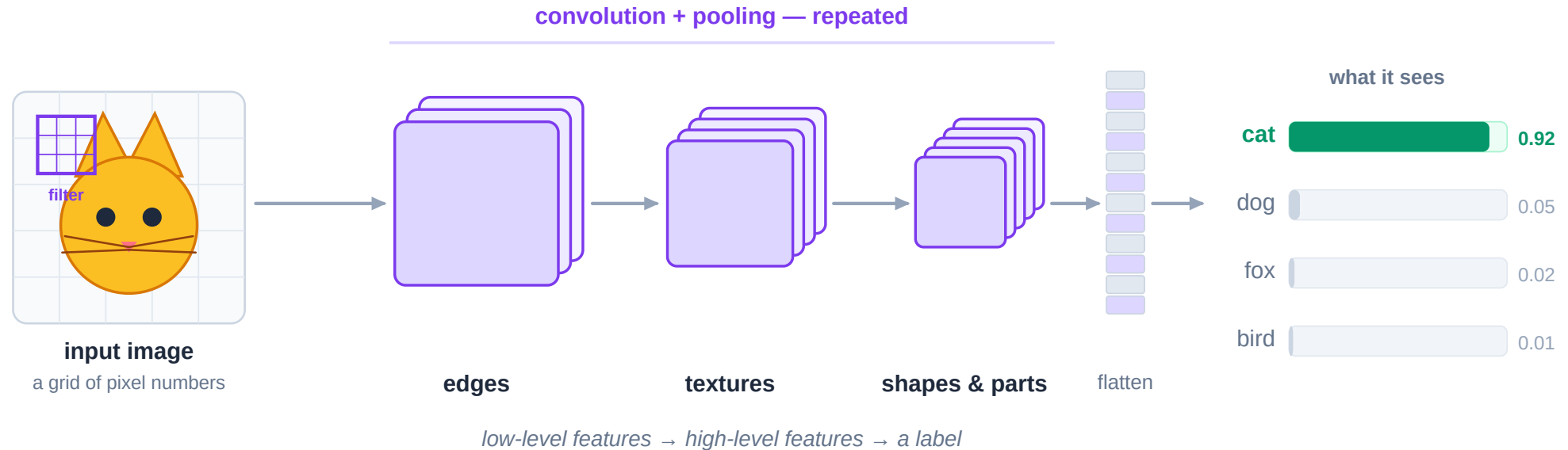
A gated memory cell that learns what to keep, forget, and output — so context survives long sequences.



● Far better at long-range memory than a plain RNN — but still strictly left-to-right, one step at a time.

Convolutional Neural Network (CNN)

Slides small learned filters across an image, building up from edges to textures to whole objects.



- Each layer slides small learned filters over the image, combining nearby pixels into ever-richer features — early layers fire on edges, deeper layers on textures and object parts, until the last layer names what it sees. Filters are shared across the whole image and every patch is processed in parallel — the architecture behind the 2012 ImageNet leap.

2010s: 2 types of neural network

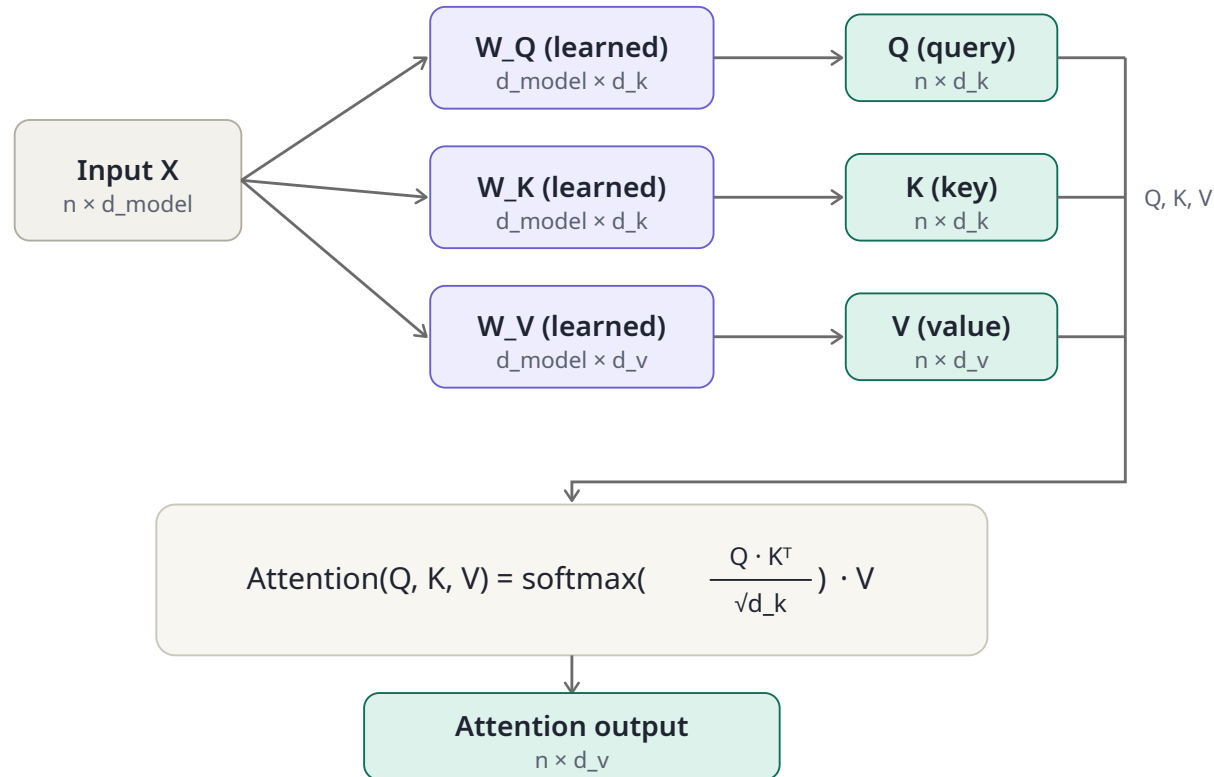
	CNN	RNN
domain	computer vision — images	language & sequences
how it reads	slides small filters across the image, all regions in parallel	one element at a time, in order
learns	local visual features: edges → textures → objects	order & context over the sequence
weakness	fixed receptive field — only a local view	slow · long-range memory fades

“Attention Is All You Need”

Vaswani et al., 2017

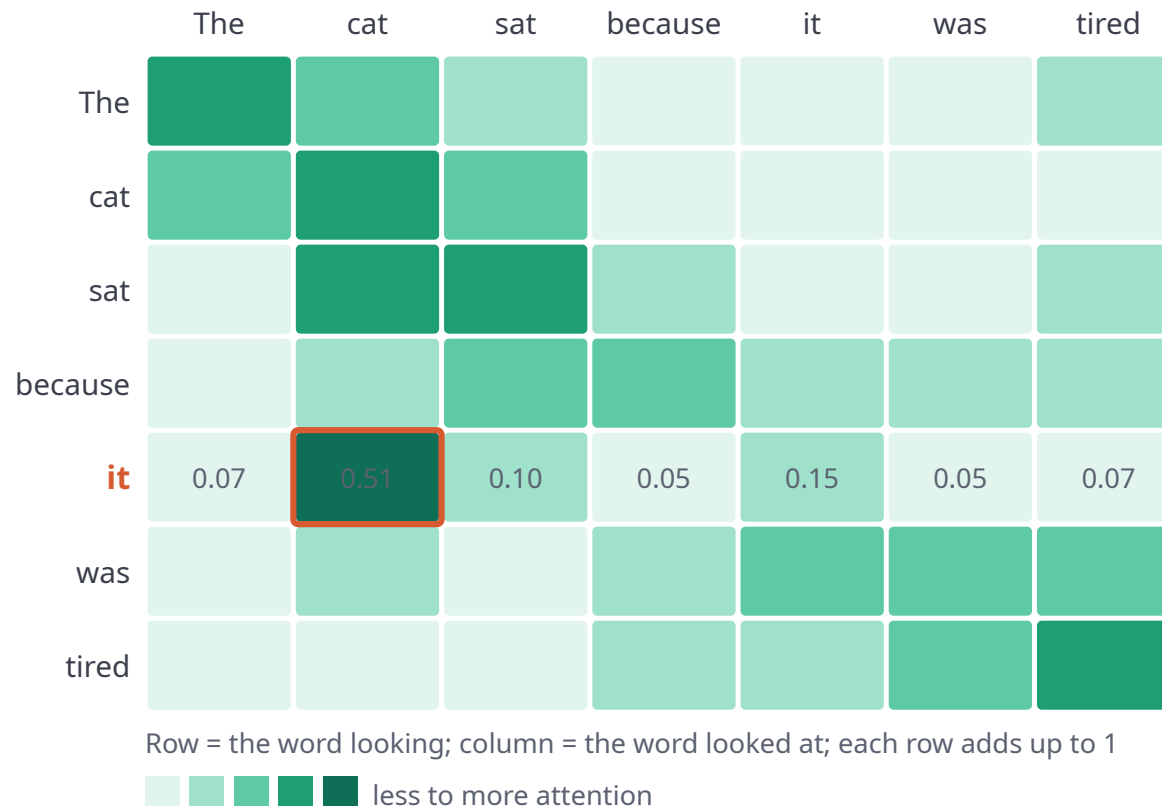
Drop recurrence. Look at **all words at once**.

Three lenses on every word



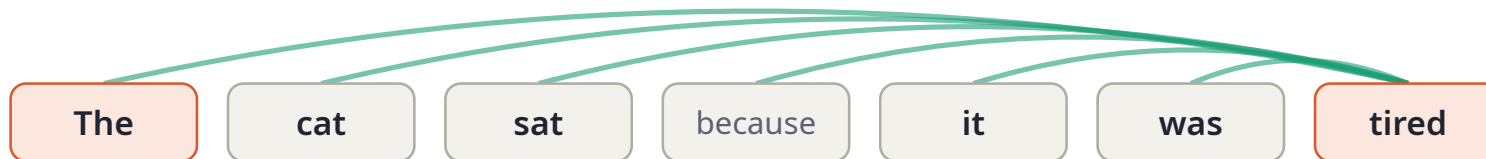
Query — what am I looking for? · **Key** — what do I offer? · **Value** — what do I carry?

Every word scores every other word

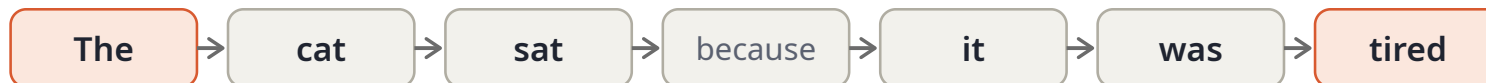


No more fading — at a price

Attention

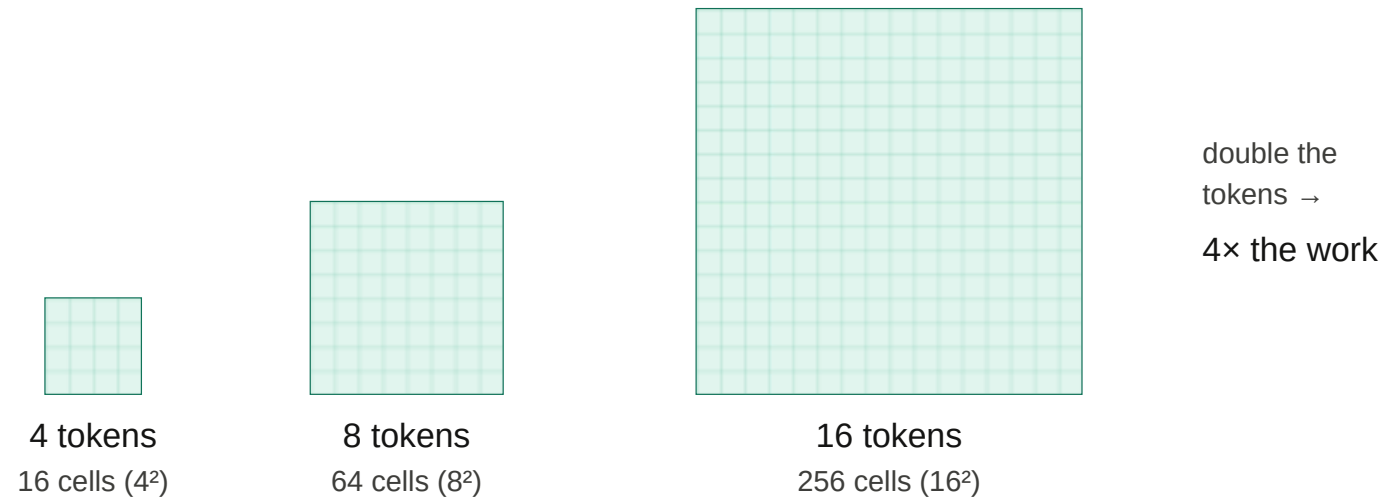


RNN / LSTM



the first word's signal fades along the chain

The price: $n \times n$



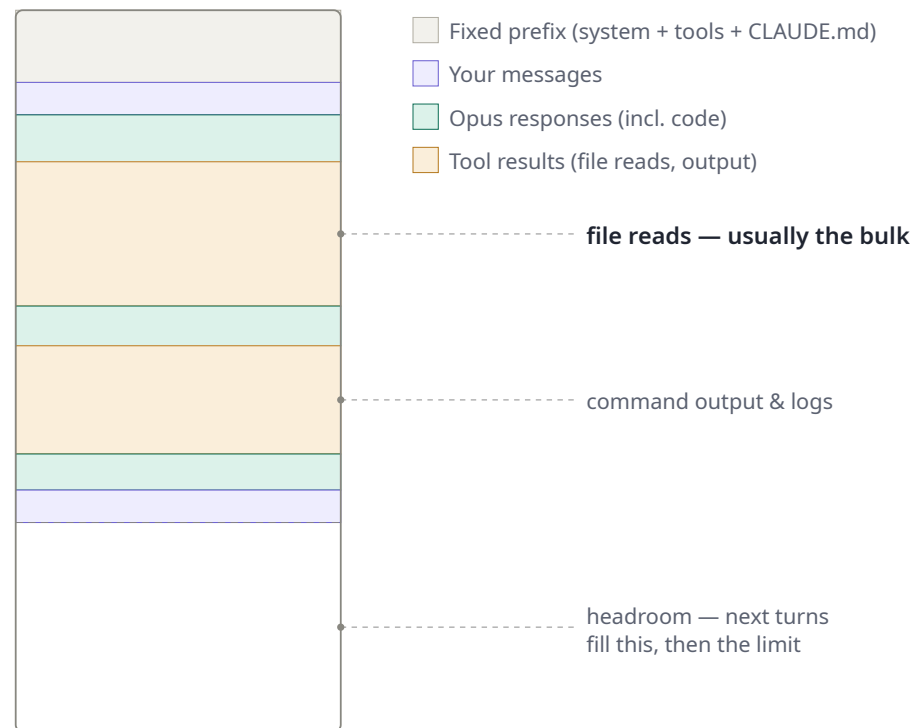
Double the tokens → **4×** the work. 1,000 tokens → 1M cells. 100,000 → **10 billion**.

The model remembers *nothing*

Every turn (every token), the **entire** conversation is re-sent:

- system prompt + tool definitions + CLAUDE.md
- your messages, the model's replies
- **every file read, every command output**

Anatomy of one request



The whole filled stack is re-sent to Opus every turn — and it keeps growing

Your levers

Lever	What it does
<code>/context</code>	see what's filling the window
<code>/compact</code>	summarise older turns — lossy
<code>/clear</code>	wipe, start fresh
<code>.gitignore</code>	keep noisy files out (Claude respects it)
lean <code>CLAUDE.md</code>	shrink the fixed per-turn tax

Part 2 — working methods

If the window is finite, the *file system* is the memory.

Docs are the contract

- Agents read `CLAUDE.md` and READMEs **at the start of every session**
- Docs are the context that **survives** `/clear`
- Unchecked agent-written docs drift — successor agents inherit the drift

Read carefully. Edit deliberately. Treat docs like code.

“Markdown is the new normal”

VS Code · *Markdown All in One* · *Markdown Preview Enhanced* · *Marp* (these slides)

Capture the *why*, not just the *what*

- **DECISIONS.md** — every non-trivial choice + **rejected alternatives**
- **Design doc before code** — captures what you *didn't* build
- The diff carries the *what*, successors inherit **files**, not conversations

One issue $\rightleftharpoons$ one PR $\rightleftharpoons$ one agent

One issue	One PR	One session
focused task	one logical change	bounded context

Small scope = the whole task **fits in the window**.

CI is the safety net

Two agents, two PRs, both pass their own tests. **Main breaks.**

Template defenses:

- migration single-head check · frontend/backend **contract check**
- e2e on single-side PRs · comment-density gate

Scoped to stay fast: typical small PR **< 2 min**, cost down ~5×.

Mechanics: where the levers live

- `.claude/skills/` & `/commands` — reusable workflows, loaded on demand
- **Subagents** — plumbing runs in a *separate* context window
- **Model choice** — Large when wrong is silent · Medium for plumbing · Smaller for cleanups

Roles & personas

Roles — *what job*: junior dev · architect · reviewer · cleaner

Personas — *how it thinks*: meticulous · creative · **maverick**

A meticulous reviewer catches the off-by-one.

A maverick reviewer asks if you should be doing this at all.

Memory & audit trails

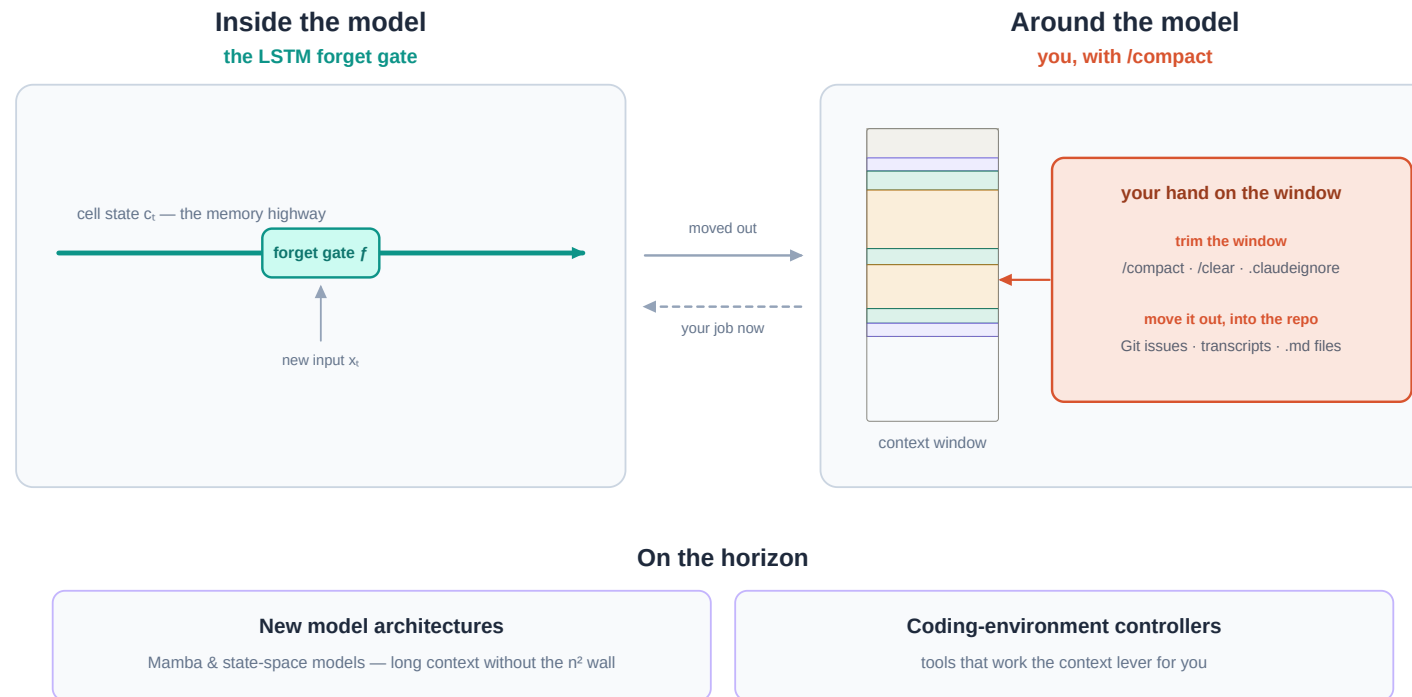
What did the agent think six months ago?

- **Conversations are ephemeral. Issues live forever.**
- Transcript → issue comment (template skill + hook)
- Agent memory: facts that survive sessions

Supervised autonomy, not autopilot

- Humans **choose the work** — the `ready` label
- Humans **approve the merge**
- Agents do the typing. **Not the deciding.**

Full circle



A finite memory forces **selective forgetting**. Sound familiar?

Takeaways

1. Attention sees everything — at n^2 cost → the window *must* be finite
2. The model is **stateless** — everything is re-sent, every turn
3. Finite window → **forgetting returns** — now it's *your* lever
4. So put the knowledge in **files, issues, and PRs** — not the conversation

Take the template: github.com/UtrechtUniversity/claude-code-template